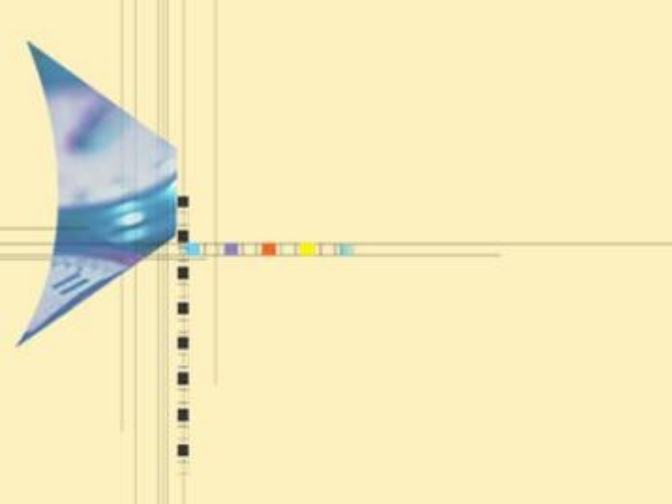




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

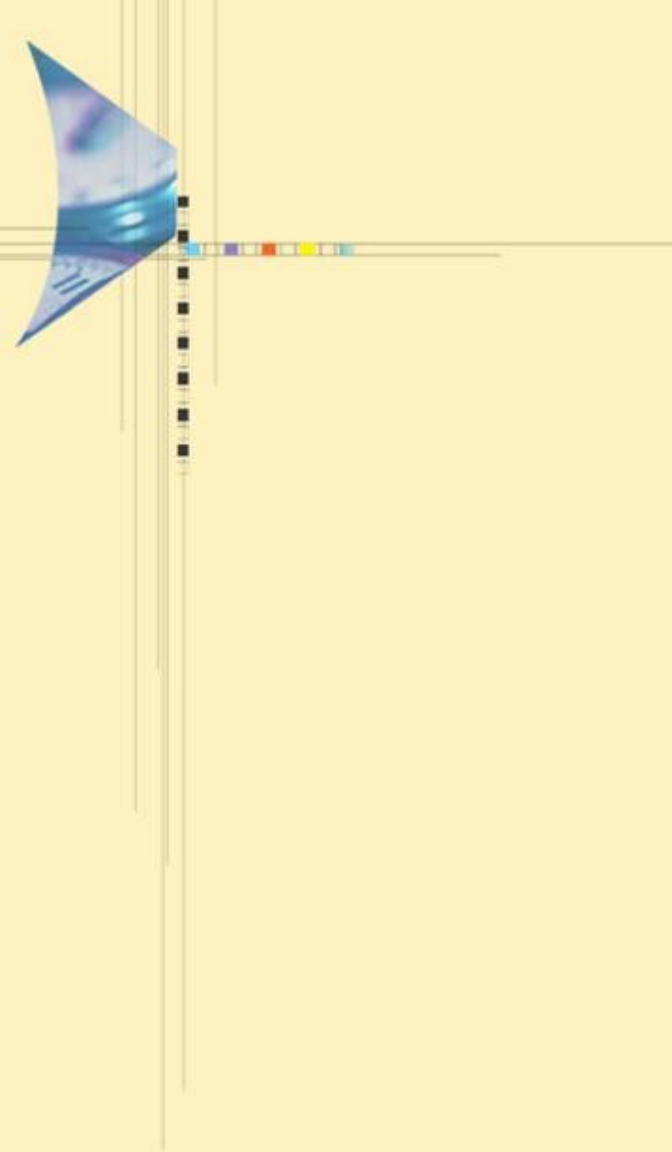
周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/2

武汉大学计算机学院

武汉大学
Wuhan University





第 六 讲

图的遍历

——Traversal of Graphs



主要内容

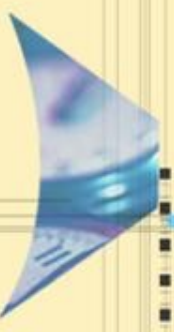
6.1 引言

6.2 深度优先搜索及应用

6.3 广度优先搜索及应用

6.4 A^* 搜索算法





6.1 引言

- 在一些诸如找出最短路径或最小生成树的图的算法中，用由它们相应的算法强加顺序访问顶点和边。
- 然而，在一些其他的算法中，访问顶点的顺序是不重要的。重要的是，不管输入的图如何，**采用一种系统的顺序来访问顶点。**
- 图遍历的两种方法：
 - 深度优先搜索
 - 广度优先搜索

6.2 深度优先搜索

- 深度优先搜索是一种强有力的遍历方法，它能帮助解决许多包含图的问题，它本质上是有根树前序遍历的一个推广。
- 假设 $G=(V,E)$ 是一个有向或无向图，系统性地遍历图 G 中每个顶点的深度优先搜索策略如下：
 - 首先所有的顶点标上未访问；
 - 选择一个起始顶点，比如说 $v \in V$ ，并标上已访问。设 w 是邻接于 v 的任意一个顶点，把 w 标为已访问；
 - 前进到另一个顶点，如说称为 x ，它是邻接于 w 并且被标为未访问的。再把 x 标为已访问，再前进到另一个邻接于 x 且标为未访问的顶点；
 - 这个选择邻接于当前顶点的一个未访问过的顶点的过程尽可能深地继续，直到找出一个顶点 y ，邻接于它的所有顶点都已被标上已访问。
 - 在这点处，我们返回到最后访问的顶点，比如说 z ，如果还有邻接于它的任何未访问顶点，就访问它。继续这种方法，我们最终返回到起始顶点。

6.2 深度优先搜索

这种遍历方法已经给出了深度优先搜索的名称，因为它在朝前(深)的方向延伸搜索。

算法 6.1 DFS

输入：有向或无向图 $G = (V, E)$ 。

输出：在相应的深度优先搜索树中对顶点的前序和后序。

1. $predfn \leftarrow 0; postdfn \leftarrow 0$
2. **for** 每个顶点 $v \in V$
3. 标记 v 未访问
4. **end for**
5. **for** 每个顶点 $v \in V$
6. **if** v 未访问 **then** $dfs(v)$
7. **end for**

过程 $dfs(v)$

1. 标记 v 已访问
2. $predfn \leftarrow predfn + 1$
3. **for** 每条边 $(v, w) \in E$
4. **if** w 标记为未访问 **then** $dfs(w)$
5. **end for**
6. $postdfn \leftarrow postdfn + 1$

6.2 深度优先搜索

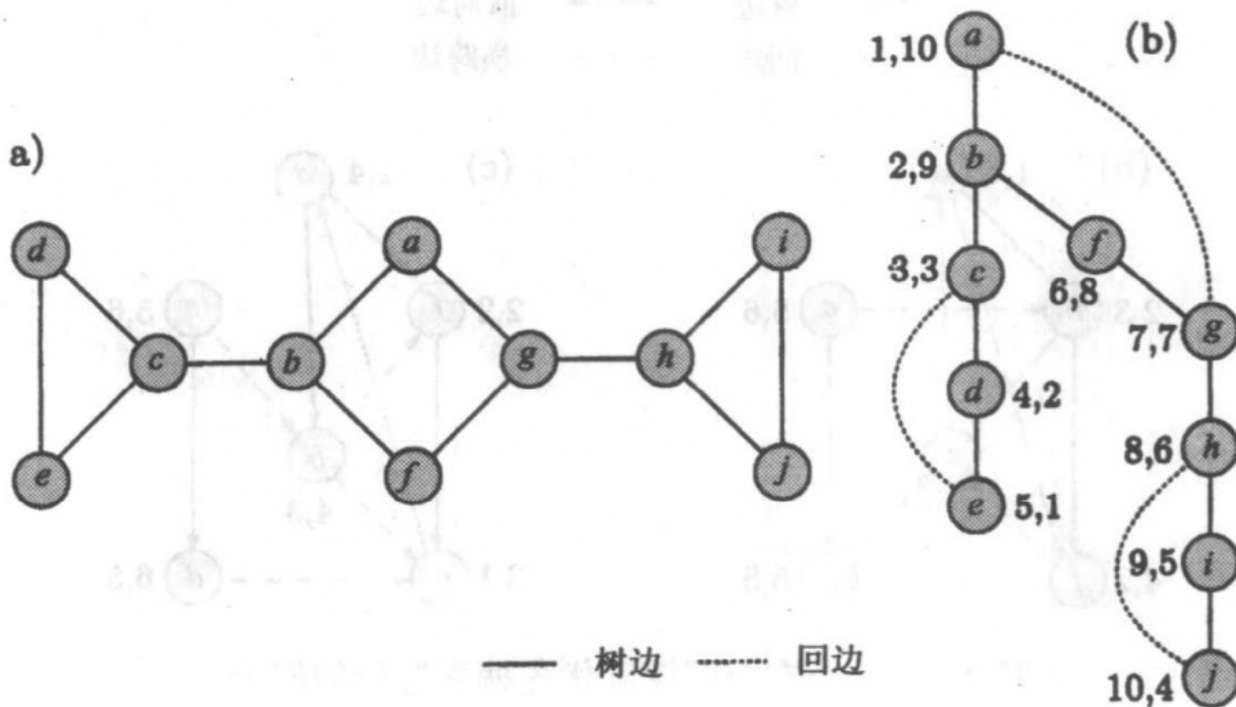
- 算法从把所有顶点标上未访问开始，它还把两个计数器predfn和postdfn 初始化为0。
- 然后，算法对 V 中每一个未访问顶点调用过程dfs，这是因为从起始点出发不是所有的顶点都可以到达的。
- 从某个顶点 $v \in V$ 开始,过程dfs通过访问 v ,把 v 标成已访问，然后递归地访问它的邻接顶点来执行对 G 的搜索。当搜索完成时,如果从起始点开始所有的顶点都可到达，就构建起一棵生成树，称为深度优先搜索生成树，它的边是在前进方向上即当探索未访问顶点时被检查的那些边。如果从起始点出发不是所有的顶点都是可到达的,那么搜索的结果产生一个生成树森林。
- 在搜索完成后，每一个顶点标上了predfn和postdfn数这两个标号，即在深度优先搜索遍历产生的生成树(或森林)的顶点上加上了前序和后序号，它们给出了访问一个顶点的开始和结束的次序。

无向图的情况

- 设 $G=(V,E)$ 是无向图，作为遍历的结果， G 的边分成以下两种类型。
 - 树边：深度优先搜索树中的边，如果在探索边 (v,w) 时， w 是首次被访问，则边 (v,w) 是树边；
 - 回边：所有其他的边。



无向图的情况



在图(a)所示的无向图上进行深度优先搜索遍历的行为

无向图的情况

- 顶点a被选做起始点，深度优先搜索树如图(b)用实线表示，虚线表示回边，在深度优先搜索树中的每一个顶点标上了两个数:predfn和postdn。
- 注意，由于顶点e有postdfn = 1,它是完成深度优先搜索的第一个顶点，又注意到，由于图是连通的，起始点用predf = 1和postdf = 10标号，10是图中的顶点数。

有向图的情况

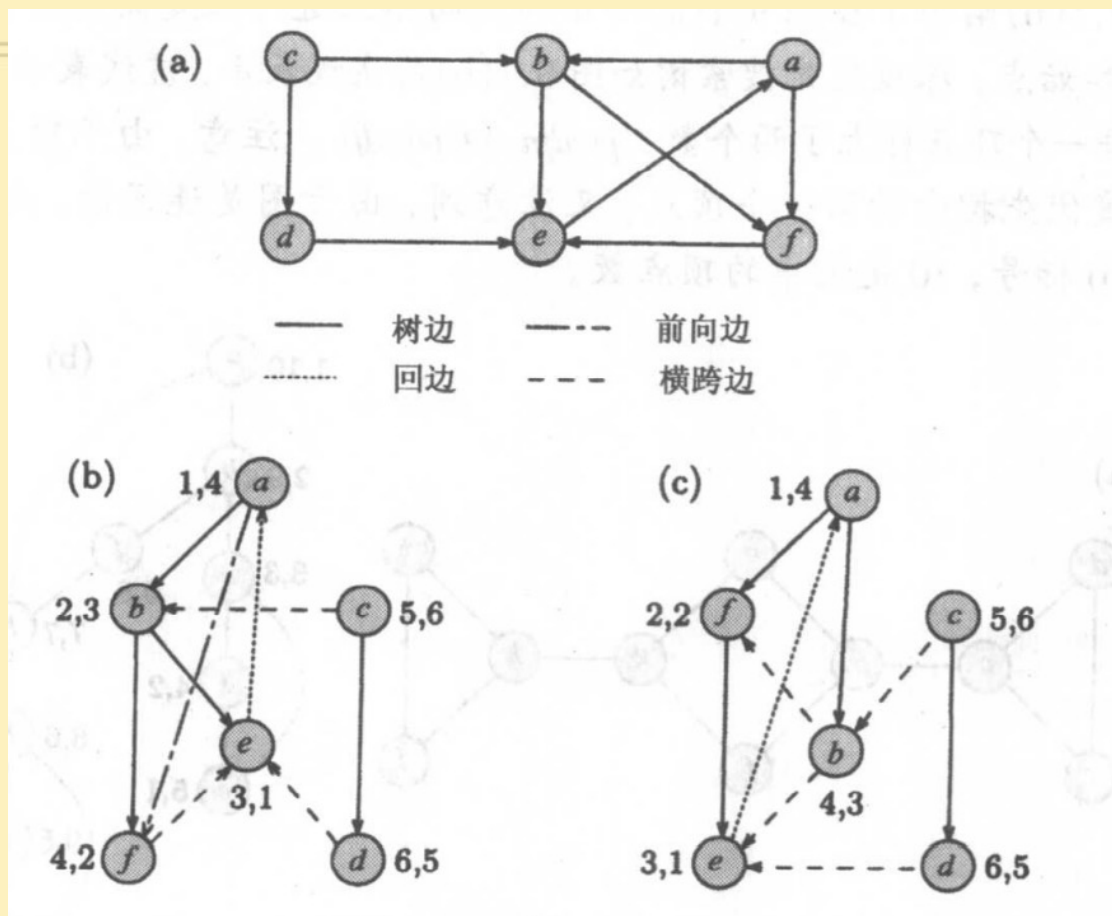
- 有向图中的深度优先搜索导致了一个或多个(有向)生成树，树的多少取决于起始点。
- 如果 v 是起始点，深度优先搜索一棵树，它由从 v 出发所有可达的顶点组成。如果那棵树没有包括所有的顶点，搜索从另一个未访问过的称为 w 的顶点再继续，构建一棵从 w 可达的所有未访问顶点组成的树。
- 这个过程继续下去直到所有的顶点都被访问过。



有向图的情况

- 在有向图的深度优先搜索遍历中， G 的边分成4种类型：
 - 树边：深度优先搜索树中的边。如果在探索边 (v,w) 时， w 是第一次被访问，则 (v,w) 是树边；
 - 回边：在迄今为止构建的深度优先搜索树中， w 是 v 的祖先，并且当探索 (v,w) 时顶点 w 已被标上已访问，这样形式的边 (v,w) 是回边；
 - 前向边：在迄今为止构建的深度优先搜索树中， w 是 v 的后裔，并且当探索 (v,w) 时顶点 w 被标上已访问，这种形式的边 (v,w) 是前向边；
 - 横跨边：所有其他的边。

有向图的情况



在图(a)中所示的有向图上进行深度优先搜索遍历的行为

有向图的情况

- 从顶点a开始，依次访问a,b,e和f。
- 当过程 dfs再次从顶点c开始，然后访问 d，再访问b后，就完成了遍历。
- 边(e,a)是一条回边，因为在深度优先搜索树中是a的后裔，并且(e,a)不是树边。
- 边(a,f)是一条前向边，因为在深度优先搜索树中，a 是f的祖先，并且(a,f)不是树边。
- 在深度优先搜索树中e和f都不互为祖先，因此边(f,e)是横跨边。
- 两条边(c,b)和(d,e)连接着两棵不同的树，它们是横跨边。
- 注意，我们可以在访问a后马上访问f而不是b，这样的话,(a,b)和(a,f)都将是树边，在这种情况下，深度优先搜索遍历的结果如图(c)所示。因此边的类型取决于顶点被访问的次序。

时间复杂性

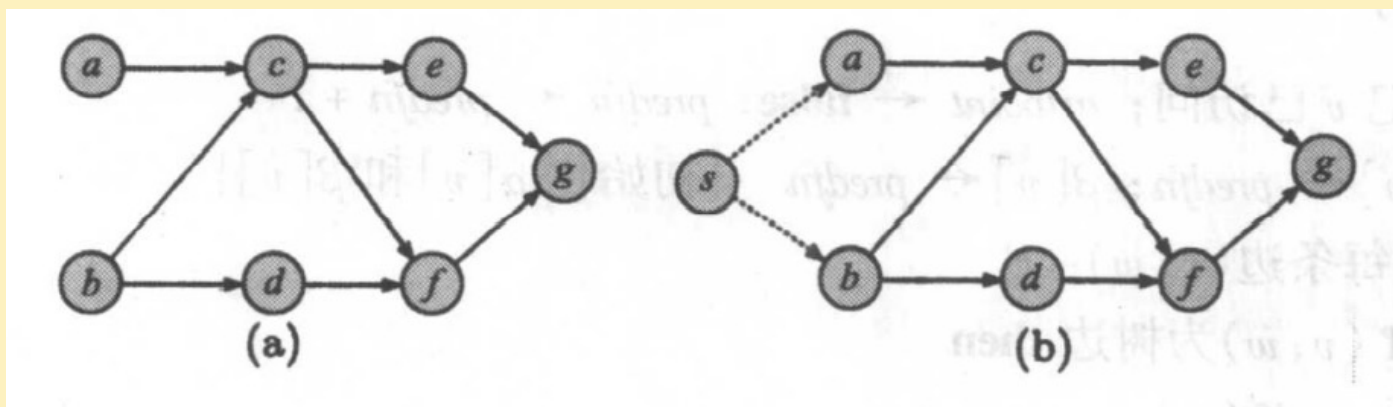
- 设图有 n 个顶点和 m 条边，图有邻接表表示。
- 算法2~4步的for循环，时间耗费是 $\Theta(n)$;
- 算法5~7步的for循环，时间耗费是 $\Theta(n)$ 。
 - 由于一旦在顶点 v 上调用过程，就将对它标上已访问标记，因此在 v 上不会有更多的调用发生，于是过程调用的次数恰好是 n 次，如果把for循环排除在外,过程调用的耗费是 $\Theta(1)$ 。因此排除for 循环后，过程调用的总耗费是 $\Theta(n)$ 。
- dfs过程中for 循环的耗费。测试顶点 w 是否标记为未访问，这一步的执行次数等于顶点 v 的邻接点数。因此这一步执行的总次数，在有向图的情况下等于它的边数，在无向图的情况下是边数的两倍。于是，不论是有向图还是无向图，这一步的耗费是 $\Theta(m)$
- 因此，算法运行的时间是 $\Theta(m+n)$ 。（图用邻接表表示，如果邻接矩阵情况呢？）

深度优先搜索的应用-图的无回路性

- 设 $G=(V,E)$ 是一个有 n 个顶点和 m 条边的有向或无向图，测试该图是否至少有一条回路。
- 为了测试 G 是否至少有一个回路，对 G 用深度优先搜索。如果在搜索过程中探测到一条回边，那么图 G 是有回路的；否则 G 是无回路的。
- 注意 G 可能不连通。如果 G 是连通无向图，那么不需要对图进行遍历，因为 G 是无回路的当且仅当它是一棵树，则当且仅当 $m=n-1$ 。

深度优先搜索的应用-拓扑排序

- 给出一个有向无回路图(dag) $G=(V,E)$, 拓扑排序问题是要找出图顶点的一个线性序, 使其满足: 如果 $(v,w) \in E$, 那么在这个序中 v 在 w 前出现。
- 例如, 在图(a)所示的有向无回路图中, 顶点的一个可能的拓扑排序是 a,b,d,c,e,f,g 。假设在这种有向无回路图中有唯一一个入度是0顶点 s 。如果没有, 可以简单地添一个新的点 s , 然后将 s 到所有入度为0的顶点之间加上边, 见图(b)。



深度优先搜索的应用-拓扑排序

- 从唯一入度为0的顶点s开始，简单地对图G执行深度优先搜索。当遍历完成时，计数器postdfn 定义了一个在有向无回路图中顶点的反拓扑序。
- 于是，为了得到这个拓扑序，可以在算法 DFS 中计数器 postdfn 刚好被增加后，加上一个输出步，把输出结果反转过来就是我们需要的拓扑序。

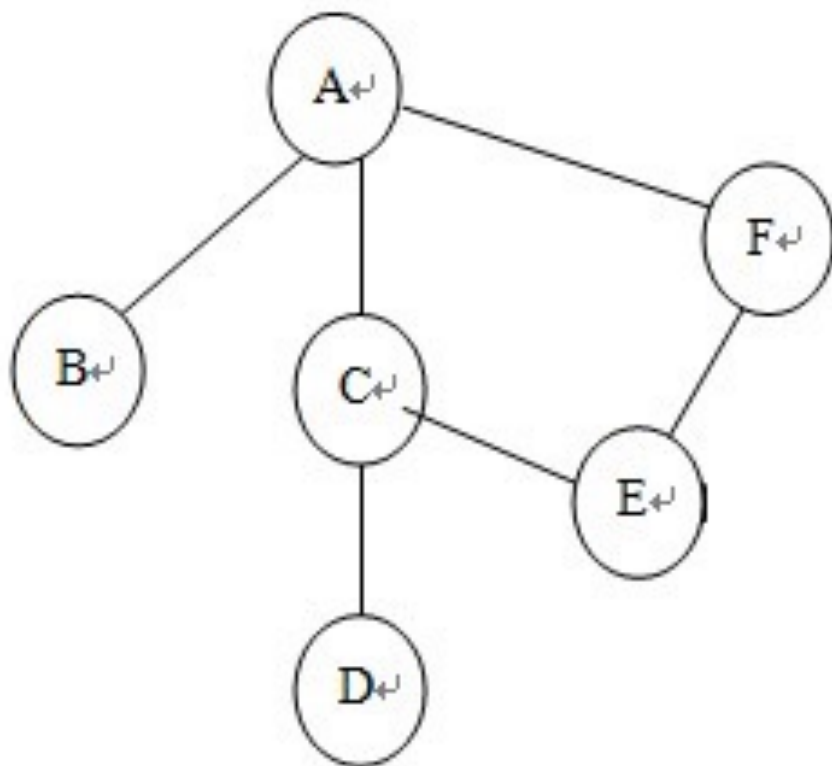
深度优先搜索的应用-寻找图的关节点

- 关节点：在多于两个顶点的无向图 G 中，存在一个顶点 v ，如果有不同于 v 的两个顶点 u 和 w ，在 u 和 w 间的任何路径都必定经过顶点 v ，则 v 称为关节点。
- 双连通图：一个图如果是连通的并且没有关节点则称为是双连通的。
- 如果 G 是连通的，移去关节 v 和与它关联的边，将产生 G 的不连通子图。
- 如何找出图的关节点？

深度优先搜索的应用-寻找图的关节点

- 为了找出关节点的集合，在图 G 上执行一个深度优先搜索遍历。对于连通图，从任一点出发深度优先遍历得到深度优先生成树，对于树中任一顶点 v 而言，其孩子节点为邻接点。由深度优先生成树可得出两类关节点的特性：
 - (1) 若生成树的根有两棵或两棵以上的子树，则此根顶点必为关节点。因为图中不存在连接不同子树顶点的边，若删除此节点，则树便成为森林。
 - (2) 若生成树中某个非叶子节点 v ，其某棵子树与 v 的祖先节点无连接，则 v 为关节点。因为删去 v ，则其子树和图的其它部分被分割开来。

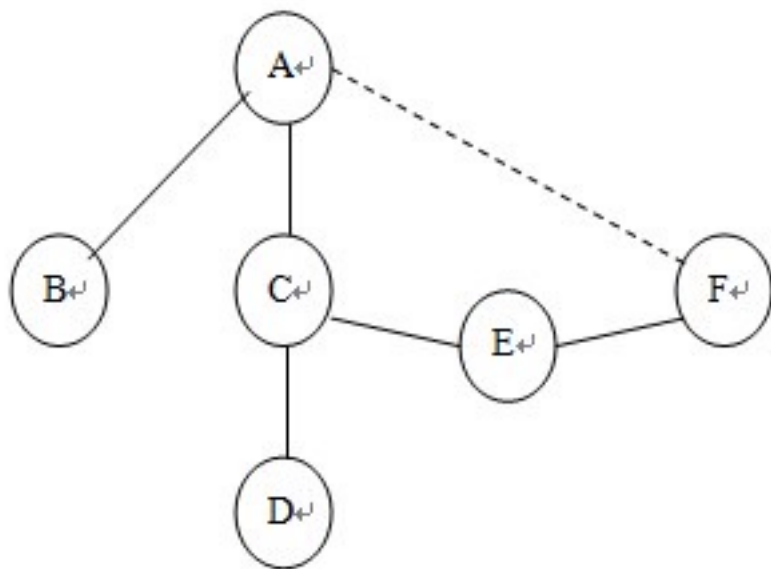
深度优先搜索的应用-寻找图的关节点



很显然去掉A或C后整个图就变成不联通了，因此A、C就是关节点。

深度优先搜索的应用-寻找图的关节点

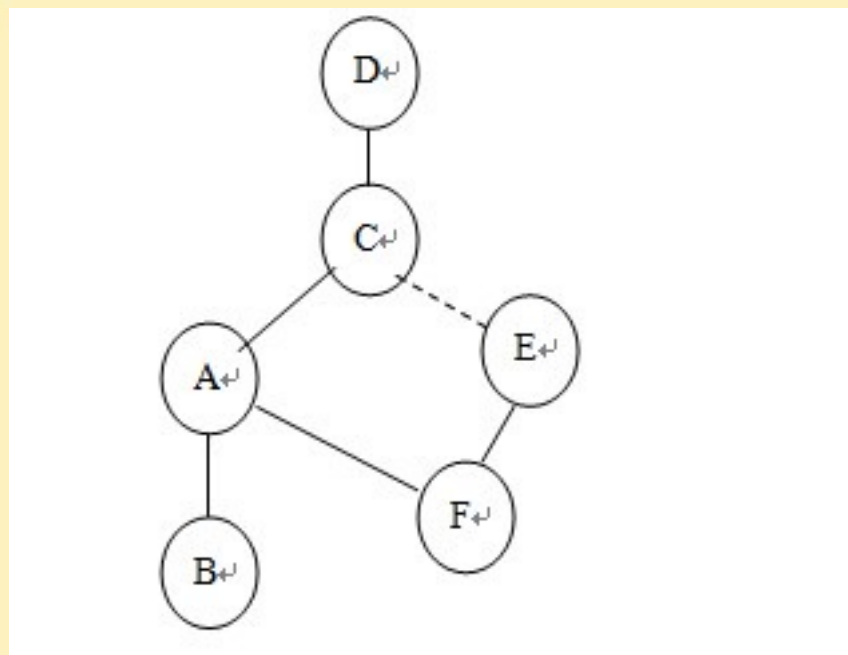
假设从A出发，搜索顺序是A, B, C, D, E, F，那么可以画出一棵搜索树：



- 根A有2棵子树分别为B，D-C-E-F。因此A必定为关节点。因为各个子树不可能存在一条到达A的祖先（A没有祖先）的回边构成一条回路。
- 所以在以某一条边开始搜索到最后搜索完毕时，需要查看根的子树的个数，若子树小于2则该点不是关节点，反之则是关节点。

深度优先搜索的应用-寻找图的关节点

假设从D出发，搜索顺序是D, C, A, B, F, E，那么可以画出一棵搜索树：



- 设当前节点是A。
- A的两棵子树分别为B，F-E；
- 因为E-F子树存在一条回边导致有一条可以到达A的祖先的回路，此时并不能确定A是不是关节点；
- 但对子树B，并没有一条回边可以到达A的祖先，当我们去掉A后，它的子树B就与它的祖先失去“联系”，因此A就是关节点。

深度优先搜索的应用-寻找图的关节点

怎么确定一条回边呢?

- 可以在dfs的过程中将搜索序列号记录下来存进 $\alpha[]$ 内, 在搜索每一条路径时结果若到达一个节点序号已经存在而且比自己的搜索序列号小, 可以将该序列号记录下来存进 $\beta[]$ 中作为某一个节点通过某一条路径所能到达的最小序列值。
- 在每一个子树回溯到该节点所能到达的最小值时, 我们先判断该值是否小于该节点的序列号, 若小于, 那么该节点可以通过该子树到达自己的祖先, 我们在将该节点的 $\beta[]$ 更新。否则, 该子树是一棵真正存在于该节点下的一颗子树。
- 总结以上陈述也就是说, 搜索每一个节点通过自己的各个子树所能到达的最小序列号是否都比自己的序列号小, 若成立则是关节点, 否则不是。

深度优先搜索的应用-寻找图的关节点

- 为了找出关节点的集合，在图G上执行一个深度优先搜索遍历。
- 在遍历的过程中，对每个顶点 v 保持两个标号： $\alpha[v]$ 和 $\beta[v]$ ， $\alpha[v]$ 只是深度优先搜索算法中的 `predfn`，每次调用深度优先搜索过程，就加1； $\beta[v]$ （通过自己的各个子树所能到达的最小序列号）初始化为 $\alpha[v]$ ，但在后来的遍历过程中可以改变，对于每一个访问的顶点，令 $\beta[v]$ 是下面几个中的最小值：
 - $\alpha[v]$ ；
 - $\alpha[u]$ ，对于每个顶点 u ， (v,u) 是回边；
 - $\beta[w]$ ，在深度优先搜索树中的每条边 (v,w) 。
- 关节点确定条件如下：
 - 根是一个关节点当且仅当在深度优先搜索树中，它有两个或更多的儿子；
 - 根以外的顶点 v 是一个关节点当且仅当 v 有一个儿子 w ，使得 $\beta[w] \geq \alpha[v]$ 。（ w 到 v 的祖先节点必须经过 v ）

算法 6.2 ARTICPOINTS

输入：连通无向图 $G = (V, E)$ 。

输出：包含 G 的所有可能关节点的数组 $A[1 \cdots \text{count}]$ 。

1. 设 s 为起始顶点
2. **for** 每个顶点 $v \in V$
3. 标记 v 未访问
4. **end for**
5. $\text{predfn} \leftarrow 0$; $\text{count} \leftarrow 0$; $\text{rootdegree} \leftarrow 0$
6. $\text{dfs}(s)$

过程 $\text{dfs}(v)$

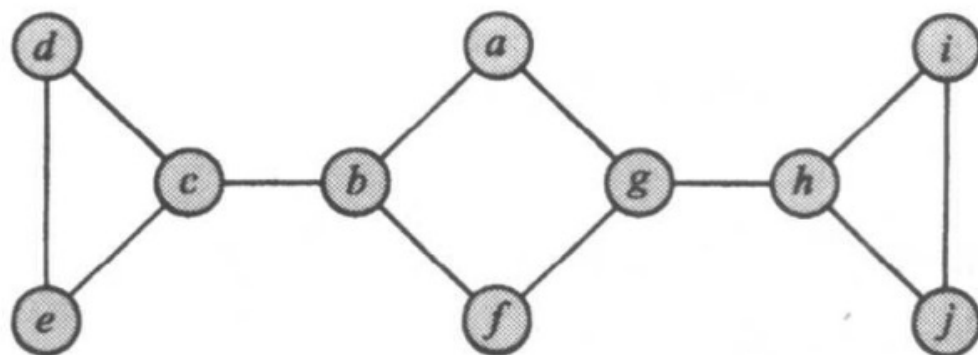
1. 标记 v 已访问; $\text{artpoint} \leftarrow \text{false}$; $\text{predfn} \leftarrow \text{predfn} + 1$
2. $\alpha[v] \leftarrow \text{predfn}$; $\beta[v] \leftarrow \text{predfn}$ {初始化 $\alpha[v]$ 和 $\beta[v]$ }
3. **for** 每条边 $(v, w) \in E$
4. **if** (v, w) 为树边 **then**
5. $\text{dfs}(w)$
6. **if** $v = s$ **then**
7. $\text{rootdegree} \leftarrow \text{rootdegree} + 1$
8. **if** $\text{rootdegree} = 2$ **then** $\text{artpoint} \leftarrow \text{true}$

count: 关节点个数

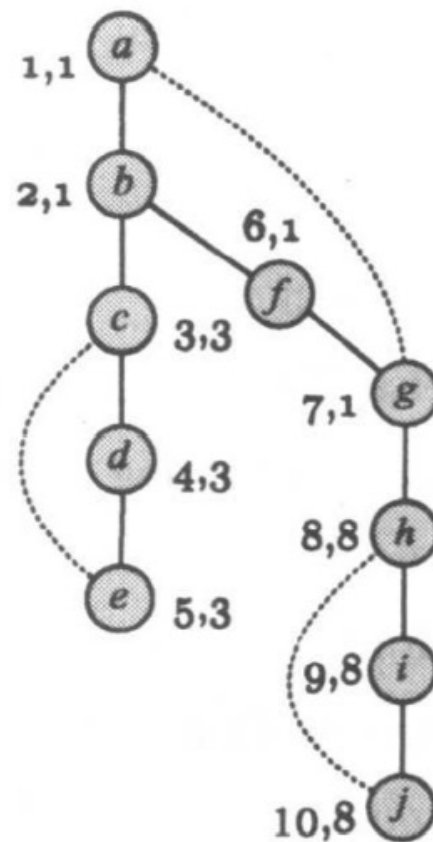
rootdegree: 树根的度

9. **else**
10. $\beta[v] \leftarrow \min\{\beta[v], \beta[w]\}$
11. **if** $\beta[w] \geq \alpha[v]$ **then** $\text{artpoint} \leftarrow \text{true}$
12. **end if**
13. **else if** (v, w) 是回边 **then** $\beta[v] \leftarrow \min\{\beta[v], \alpha[w]\}$
14. **else** do nothing { w 是 v 的父亲 }
15. **end if**
16. **end for**
17. **if** artpoint **then**
18. $\text{count} \leftarrow \text{count} + 1$
19. $A[\text{count}] \leftarrow v$
20. **end if**





—— 树边 回边



关节点: **c, b, h, g**

深度优先搜索的应用-强连通分支

- 给出一个有向图 $G=(V,E)$ ， G 中的强连通分支是它顶点的极大集。在该集中，每一对顶点间都存在一条路径。
- 算法STRONGCONNECTCOMP用深度优先搜索来确定在有向图中的所有的强连通分支。

深度优先搜索的应用-强连通分支

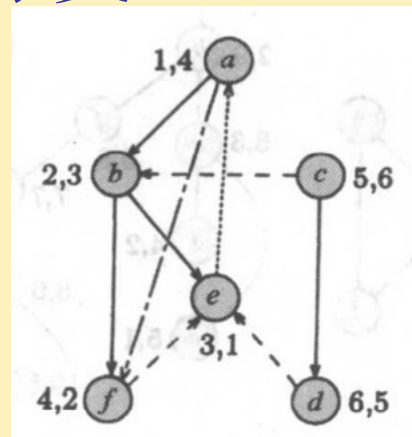
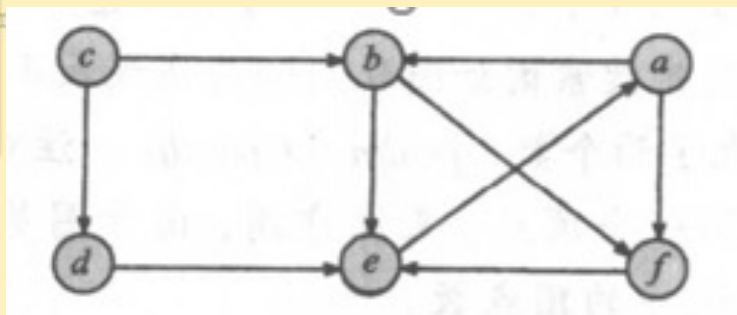
STRONCCONNECTCOMP 算法

输入:有向图 $G=(V,E)$ 。

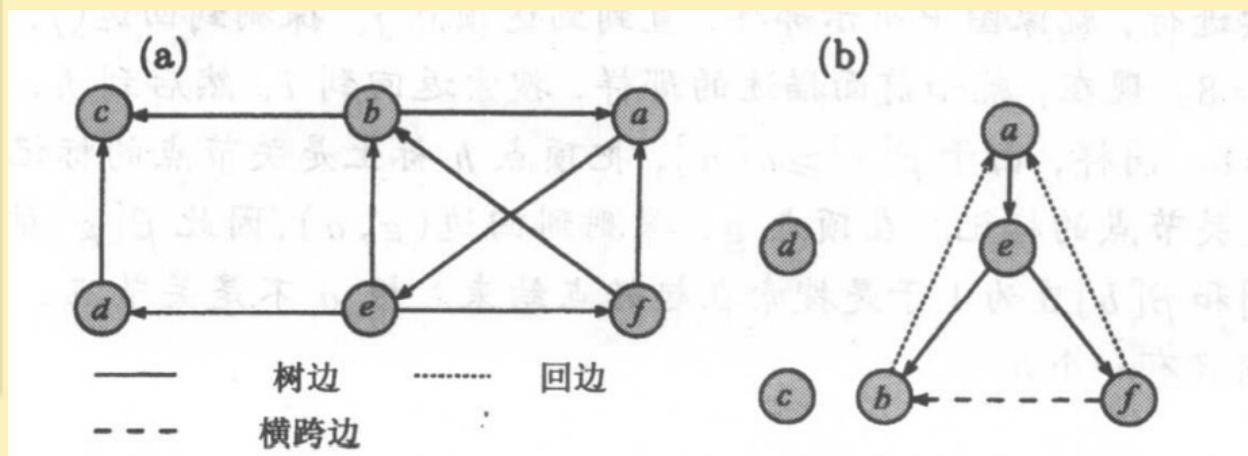
输出: G 中的强连通分支

1. 在图 G 上执行深度优先搜索，对每一个顶点赋给相应的 $postdfn$ 值。
2. 颠倒图 G 中边的方向，构成一个新的图 G' 。
3. 从具有最大 $postdfn$ 数值的顶点开始，在 G' 上执行深度优先搜索，如果深度优先搜索不能到达所有的顶点，则在余下的顶点中找一个 $postdfn$ 数值最大的顶点开始下一次深度优先搜索。
4. 在最终得到的森林中的每一棵树对应一个强连通分支。

深度优先搜索的应用-强连通分支



图中顶点的后序: e, f, b, a, d, c



三个强连通分支:
d
c
a, b, e, f

6.3 广度优先搜索

- 与深度优先搜索不同，在广度优先搜索中，当访问了一个顶点 v 后，接下来访问邻接于 v 的所有顶点，产生出来的树称为广度优先搜索树。
- 这种遍历的方法可以通过用一个**队列**存储没有检查过的顶点来实现。
- 广度优先的算法BFS能够用到有向和无向图。
- 初始的时候，所有的顶点都标上未访问，计数器bfn初始为0，它表示顶点从队列中移出的次序。
- 在无向图的情况下，一条边或者是树边或者是横跨边；
- 如果图是有向的，那么边可以是树边、回边或横跨边，不存在前向边。

6.3 广度优先搜索

算法 6.3 BFS

输入：有向或无向图 $G = (V, E)$ 。

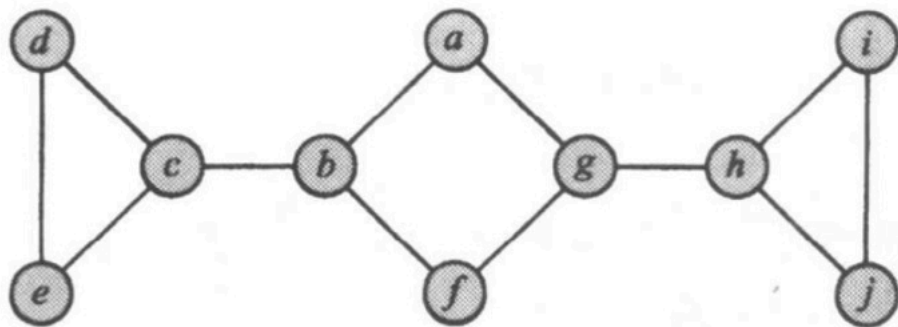
输出：广度优先搜索次序中顶点的编号。

1. $bfn \leftarrow 0$
2. **for** 每个顶点 $v \in V$
3. 标记 v 未访问
4. **end for**
5. **for** 每个顶点 $v \in V$
6. **if** v 标记为未访问 **then** $bfs(v)$
7. **end for**

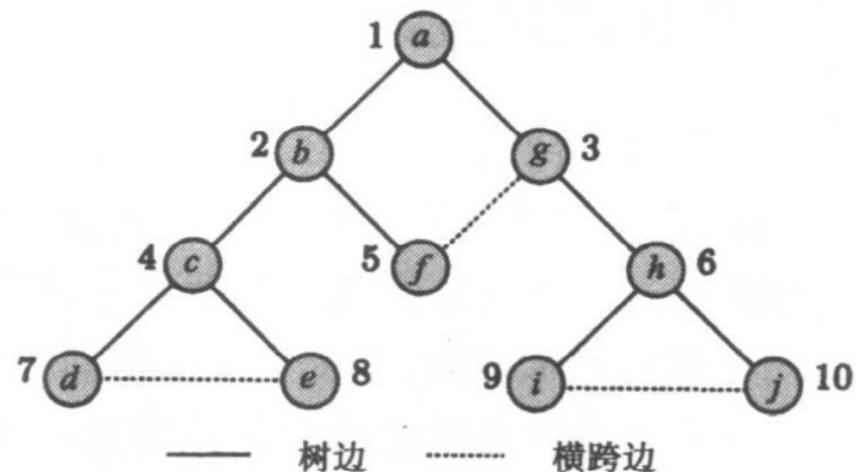
过程 $bfs(v)$

1. $Q \leftarrow \{v\}$
2. 标记 v 已访问
3. **while** $Q \neq \{\}$
4. $v \leftarrow Pop(Q)$
5. $bfn \leftarrow bfn + 1$
6. **for** 每条边 $(v, w) \in E$
7. **if** w 标记为未访问 **then**
8. $Push(w, Q)$
9. 标记 w 已访问
10. **end if**
11. **end for**
12. **end while**

6.3 广度优先搜索



a



b

把广度优先搜索遍历用到图(a)所示的图上，并以顶点a为起始点时的运作情况

6.3 广度优先搜索

- 在弹出顶点a后，就把顶点b和g压入队列并且标上已访问，
- 接着把顶点b从队列中移出，并且把它的还没有被访问过的邻接点c和f压入队列，标上已访问。
- 这个把顶点压入队列以后又把它们从队列中移出来的过程一直继续，直到顶点j最终从队列中移出。
- 这时，队列变空，广度优先搜索遍历也完成了。
- 在图中，每一个顶点标上了它的 bfn 数，这就是该顶点从队列中移出的次序。

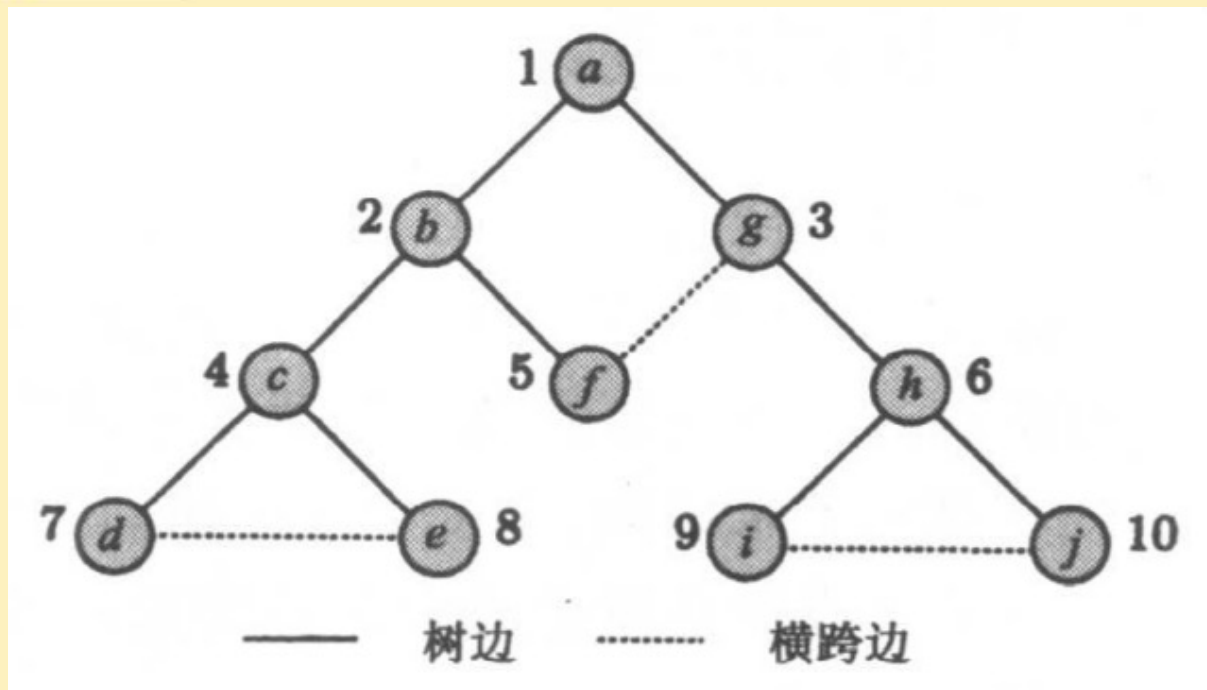
算法时间复杂度分析

- 广度优先搜索应用在具有 n 个顶点、 m 条边的图(包括有向图和无向图)上时, 它的时间复杂性是和深度优先搜索一样的, 为 $\Theta(n+m)$;
- 如果图是连通的或者 $m \geq n$, 那么时间复杂性可以简化为 $\Theta(m)$ 。

广度优先搜索的应用-求最短距离

- 广度优先搜索的应用在图和网络算法中是很重要的。
- 例如：设 $G=(V,E)$ 是连通无向图， s 是 V 中的一个顶点，要找出从 s 到其他每一顶点的距离，这里从 s 到一个顶点 v 的距离定义为：从 s 到 v 的任意路径的最少边数。
- 当把算法BFS用到 G 上并以 s 作为起始点时，产生的广度优先搜索树中从 s 点到其他任意顶点的路径有最少的边数。
- 于是上面的问题可以很容易做到。只要在每个顶点压入队列前，标上它的距离就可以了。这样，起始点将标上0,它的邻接点是1,依次类推。很明显，每个顶点的标号是它到起始点的最短距离(自己练习)。

广度优先搜索的应用-求最短距离



- 例如在上图中，顶点a将被标上0，顶点b和g标上1，顶点c,f和h标上2，最后，顶点d,e,i和j标上3。

6.4 A*算法

- A*算法是在1968年由Peter E. Hart、Nils J. Nilsson和Bertram Raphael共同提出的一种启发式搜索算法。用于解决路径找寻和图的遍历问题。“A”代表该算法是在一系列算法中的一个，而星号“*”象征着该算法在启发信息的加权之下，能够保证找到一条最优路径，即“最佳路径”（best possible path）。
- A*算法结合了最佳优先搜索（通过启发式方法优先探索可能更接近目标的节点，贪心算法）和最小成本策略（寻找起点到当前点的最短路径，Dijkstra算法），以求在保证最短路径的同时减少搜索量。
- 它通过一个估价函数 $f(n)$ 来评估哪些路径最有可能导向目标节点，其中 n 表示当前的节点。

A*算法原理

- 估价函数 $f(n)$ 的构成包括两部分，即： $g(n)$ 和 $h(n)$ 。
 - $g(n)$ 表示从起点到当前节点 n 的实际成本（走过的路）；
 - $h(n)$ 是一个启发式函数，用于估计从节点 n 到目标节点的最低成本，也称为启发式成本（未来的路）；
 - 因此， $f(n) = g(n) + h(n)$ 。
- 估价函数的设计是A*算法效率的关键所在。如果 $h(n)$ 始终小于或等于节点 n 到目标节点的实际成本，则A*算法保证能找到最短路径。

A*算法和A算法区别

1. 启发式函数的设计:

- **A算法**: 仅使用启发式函数 $h(n)$ 来评估路径成本, 其中 $h(n)$ 表示从当前节点 n 到目标节点的估计代价。这种设计可能导致算法偏向于启发式评估较低的路径, 但不一定能找到最短路径。
- **A*算法**: 在启发式函数 $h(n)$ 的基础上, 增加了实际路径成本 $g(n)$, 即从起点到当前节点 n 的实际代价。其评估函数为 $f(n)=g(n)+h(n)$ 。这种设计使得A*算法能够更全面地评估路径成本, 从而在启发式函数满足一致性条件时, 保证找到最短路径。

2. 搜索效率

- **A算法**: 由于仅依赖启发式函数 $h(n)$, 可能会扩展大量与目标节点无关的节点, 导致搜索效率较低。
- **A*算法**: 通过结合实际路径成本 $g(n)$ 和启发式函数 $h(n)$, 能够优先扩展与目标节点更接近的节点, 从而减少搜索空间, 提高效率。

3. 完备性与最优性

- **A算法**: 虽然能够找到解决方案, 但不一定能保证找到最短路径, 尤其是在启发式函数不够准确时。
- **A*算法**: 在启发式函数满足一致性 (即 $h(n)$ 不超过实际代价) 时, A*算法不仅能够找到解决方案, 还能保证该解决方案是最优的。

A* 算法

算法 A-star

输入：一个图或者网格（含有节点和边），起点start，目标点goal

输出：起点到目标点的最短路径（如果存在）

//初始化

1. 创建一个开放列表OpenList，初始只含有起点 **//存储待评估的节点，**

2. 创建一个关闭列表ClosedList，初始为空表 **//存储已评估的节点**

计算起点s的估计代价值： $f(\text{start})=g(\text{start})+h(\text{start})=0+h(\text{start})$

3. **While** OpenList 不为空 **do** **// 循环**

4. current $\leftarrow$ OpenList中估计代价值 $f(n)$ 值最小的节点

5. **if** current=goal **then**

6. 停止搜索，重建路径并返回 **// 表示已经达到目标**

7. **end if**

8. 将 current 从OpenList中删除 **//将当前节点从开放列表移动关闭列表中**

9. 将 current 添加到 ClosedList中

下一页

//处理所有邻居节点

```
10.  for each neighbor of current  //遍历current的所有邻近节点
11.      if neighbor in ClosedList then
12.          continue  //跳过，进入下一轮for循环选择下一个邻居节点
13.      end if
14.      tentative_g = g(current) + c(current, neighbor)  //计算暂时的g分数值, c
为边的代价
15.      if neighbor 不在 OpenList 中 then
16.          将neighbor 加入 OpenList
17.          tentative_is_better ← true
18.      else if tentative_g < g(neighbor) then
19.          tentative_is_better ← true
20.      else
21.          tentative_is_better ← false
22.      end if
23.      if tentative_is_better = true then  //目前最好路径
24.          parent(neighbor) ← current  //将current 设为neighbor父节点
25.          g(neighbor) ← tentative_g  //更新neighbor的g分数
26.          f(neighbor) ← g(neighbor)+h(neighbor) //更新neighbor的f分数
27.      end if
28.  end for
29. end while
30. return None  //没有找到路径
```

启发式函数 $h(n)$ 的选择

启发式函数 $h(n)$ 是A*算法的关键，它应该估计从当前节点到目标节点的代价，并满足：

- (1) 启发函数必须低估真实代价(Admissibility)
- (2) 启发函数与实际代价保持一致性(Consistency)

应用A*算法时，通常需要根据具体问题调整启发式函数 $h(n)$ ，以适应不同环境下的最优化搜索。常用的启发函数有：

- 曼哈顿距离（适用于网格路径）
- 欧几里得距离（适用于自由空间路径）
- 对角距离（切比雪夫距离，适用于可对角移动）

i点坐标(x1,y1), j点坐标(x2,y2)

- 欧氏距离:

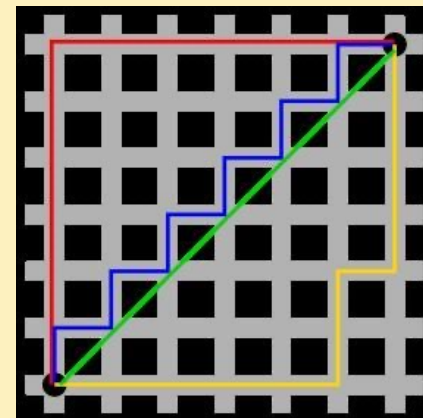
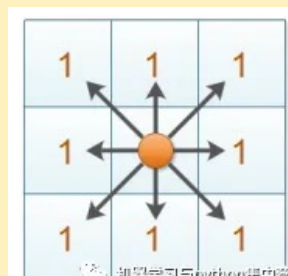
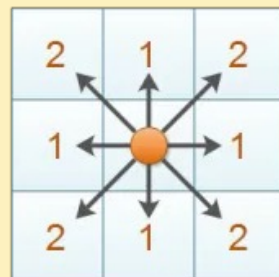
$$d(i,j)=\sqrt{(x1-x2)^2+(y1-y2)^2}$$

- 曼哈顿距离为:

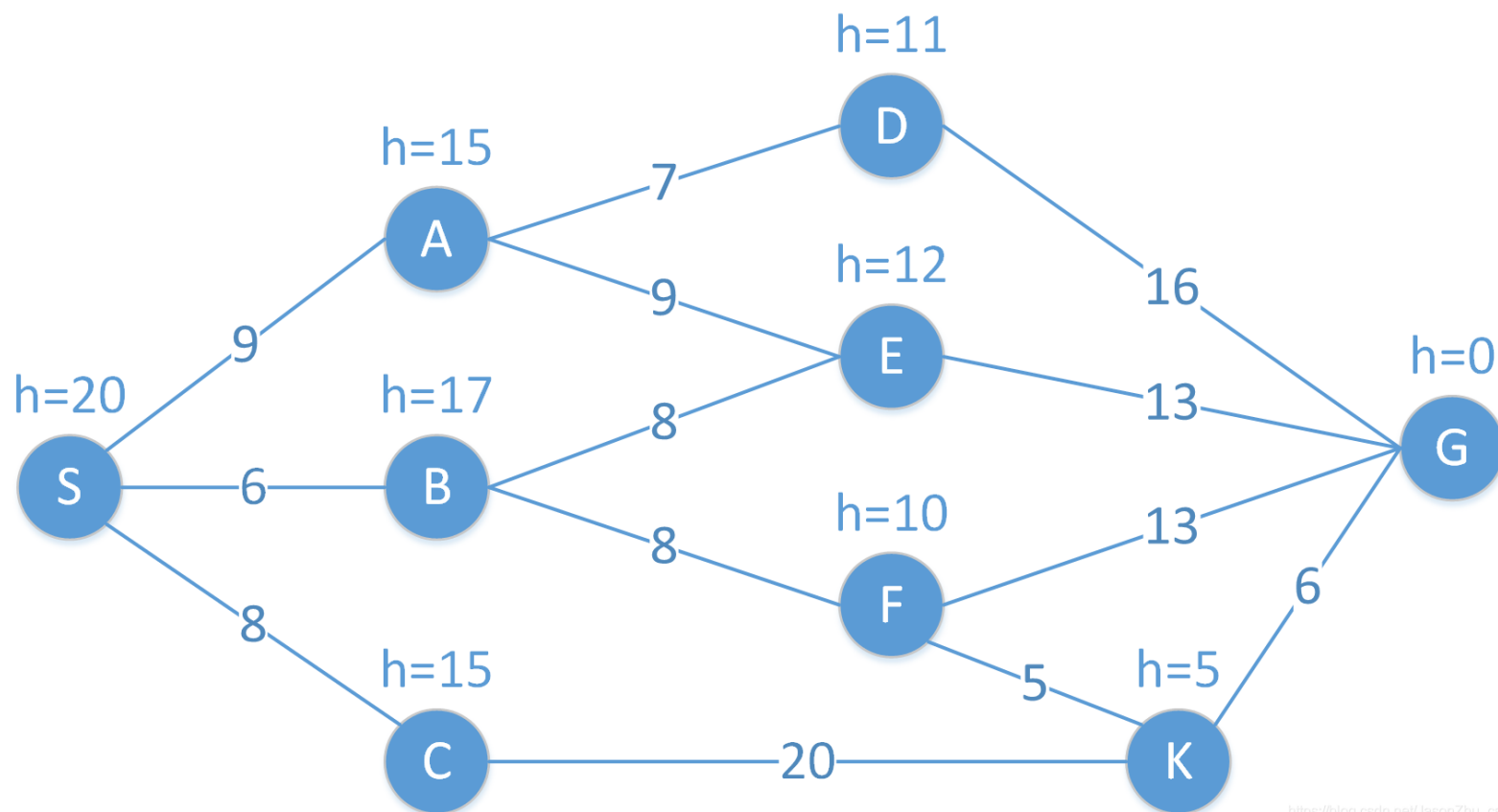
$$d(i,j)=|x1-x2|+|y1-y2|$$

- 对角距离:

$$d(i,j)=\max(|x1-x2|, |y1-y2|)$$



实例

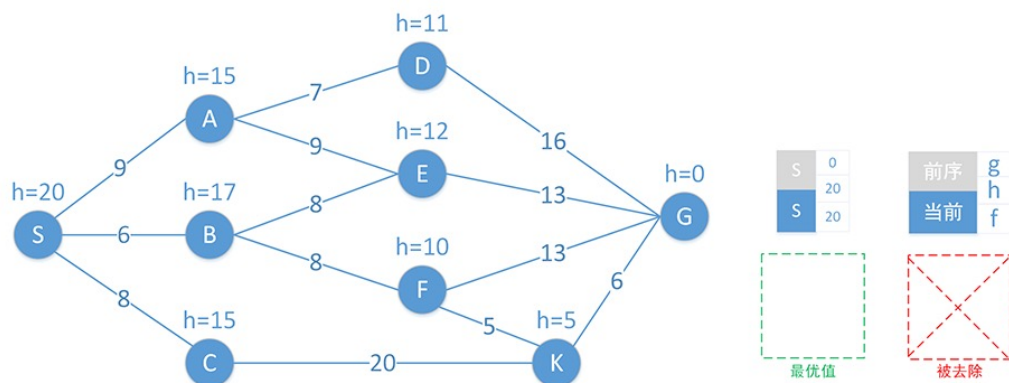


https://blog.csdn.net/JasonZhu_csdn



S为起始 (start) 节点, G为目标 (goal) 节点。

- 节点之间连线是两点的路径长度, 如A到E的路径长度 $c(A,E) = 9$ 。
- 节点旁的h值是当前节点到达目标节点 (G) 的预估值, 如 $h(A)=15$, 表示从当前点A到达目标点G的估计路径长度为15, 此处 $h(x)$ 即为启发函数。
- 从起点S到达当前节点x的路径长度表示为 $g(x)$ 。
- 从起点S到达目标G并经过点x的估计距离长度表示为 $f(x) = g(x) + h(x)$, 该公式是A*算法的核心公式。
- A*算法通过不断的选择估计距离f最小的节点, 逐渐构建最短路径。



- 灰色为前序节点
- 蓝色当前节点x
- 绿色框为当前OPEN集合中的最优节点
- 红色框表示当前OPEN集合中需要被删除的节点
- 在OPEN、CLOSED中每一行表示一次完整迭代完成时两集合中的节点集合。



- 最后的最优路径是: S->B->F->K->G
- 注: 当两个节点f相同时, h小的更优

A*算法的特例

- 如果取 $f(n)=d(n)$, 则它将退化为广度优先搜索。这里 $d(n)$ 是从起点到 n 经过的步数(n 在BFS搜索树中的层次数))
- 如果取 $f(n)=g(n)$, A*算法退化为Dijkstra算法的特例 (基于代价的广度优先搜索)
- 如果取 $f(n)=h(n)$, A*算法就是基础A算法

A*算法应用

- A*算法广泛应用于各类游戏和导航系统中，用于寻找最短行走路径或者驾车路线。例如，计算机游戏中的非玩家角色（NPC）运动路径规划，地图软件中的最优驾车或步行路线推荐等。
- 应用A算法时，通常需要根据具体问题调整启发式函数 $h(n)$ ，以适应不同环境下的最优化搜索。